

Es hora de que pongas en práctica todo lo aprendido. 

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 

¡Manos a la obra!

1. Introducción

Deberemos realizar los **Test Unitarios** para las siguientes clases:

```
public class TextoCool {

    public String capitalizarTexto(String texto) {
        texto = texto.toLowerCase();
        String[] palabras = texto.split(" ");
        StringBuilder resultado = new StringBuilder();
        for (String palabra : palabras) {
            String capitalizada = palabra.substring(0,
1).toUpperCase() + palabra.substring(1);
            resultado.append(capitalizada).append(" ");
        }
        return resultado.toString().trim();
    }

    public String coolNickName(String nickName) {
        return "__--*" + nickName.toUpperCase() + "*--__";
    }

    public String mayusculasMinusculas(String texto) {

        StringBuilder resultado = new StringBuilder();
        boolean mayuscula = true;
```

```

for (char c : texto.toCharArray()) {
    if (Character.isLetter(c)) {
        if (mayuscula) {
            resultado.append(Character.toUpperCase(c));
        } else {
            resultado.append(Character.toLowerCase(c));
        }
        mayuscula = !mayuscula;
    } else {
        resultado.append(c);
    }
}

return resultado.toString();
}
}

```

```

public class Matematica {

    public double raizCuadrada(double numero) {
        return Math.sqrt(numero);
    }

    public double potencia(double base, int exponente) {
        return Math.pow(base, exponente);
    }

    public int sumar(int a, int b) {
        return a+b;
    }

    public int restar(int a, int b) {
        return a-b;
    }

    public double dividir(int dividendo, int divisor) {
        return dividendo/divisor;
    }
}

```

```
public int multiplicar(int a, int b) {  
    return a*b;  
}  
  
public String parImpar(int num) {  
    if (num % 2 == 0) {  
        return "Par";  
    } else {  
        return "Impar";  
    }  
}  
}
```

2. Desafío 🎯

Realizar tantos `@Tests`, como funciones hayan. En el caso de animarse, es posible colocar más de un `assertEquals` o `assertNotEquals` por `@Test`.

📌 **Nota:** Usar `BeforeAll` y `AfterAll`

3. ¿Dónde se lleva a cabo? 🏠

Eclipse, usando JUnit5.

4. Tiempo de dedicación ⌚

2 horas

5. Plus ➕

Tomar ejercicios de las lecciones anteriores y agregarles mayor cantidad posible de tests.

6. ⚠️ Condición

Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío

A continuación te presentamos una de las posibles soluciones al caso anterior:

```
class MatematicaTest {  
  
    public static Matematica matematica;  
  
    @BeforeAll  
    public static void configuracionInicial() {  
        matematica = new Matematica();  
    }  
  
    @Test  
    void sumar() {  
        int expected = 4;  
        int actual = this.matematica.sumar(2, 2);  
        assertEquals(expected, actual);  
    }  
  
    @Test  
    void restar() {  
        int expected = 4;  
        int actual = this.matematica.restar(10, 6);  
        assertEquals(expected, actual);  
    }  
  
    @Test  
    void dividir() {  
        double expected = 2;  
        double actual = this.matematica.dividir(8, 4);  
        assertEquals(expected, actual, 0.001);  
    }  
  
    @Test  
    void multiplicar() {
```

```

        int expected = 6;
        int actual = this.matematica.multiplicar(3, 2);
        assertEquals(expected, actual);
    }

    @Test
    void parImpar() {
        String expected = "Par";
        String actual = this.matematica.parImpar(2);
        assertEquals(expected, actual);

        expected = "Impar";
        actual = this.matematica.parImpar(1);
        assertEquals(expected, actual);
    }

    @Test
    void potencia() {
        double expected = 27;
        double actual = this.matematica.potencia(3,3);
        assertEquals(expected, actual, 0.001);
    }

    @Test
    void raizCuadrada() {
        double expected = 8;
        double actual = this.matematica.raizCuadrada(64);
        assertEquals(expected, actual, 0.001);
    }
}

```

```

class TextoCoolTest {

    public static TextoCool textoCool;

    @BeforeAll

```

```

    public static void configuracionInicial() {
        textoCool = new TextoCool();
    }

    @AfterAll
    public static void limpieza() {
        textoCool = null;
    }

    @Test
    void capitalizarTexto() {
        String expected = "Chiquito Fernando Del Valle";
        String actual = this.textoCool.capitalizarTexto("chiquito fernando
del valle");

        assertEquals(expected, actual);
    }

    @Test
    void coolNickName() {
        String expected = "__--**CHIQUITO**--__";
        String actual = this.textoCool.coolNickName("Chiquito");

        assertEquals(expected, actual);
    }

    @Test
    void mayusculasMinusculas() {
        String expected = "HoLa, CoMo EsTaS?";
        String actual = this.textoCool.mayusculasMinusculas("Hola, como
estas?");

        assertEquals(expected, actual);
    }
}

```